

MCG 5353 Robotics Project 2

Panda Gazebo Color Sorting

Weight: 15% of course grade

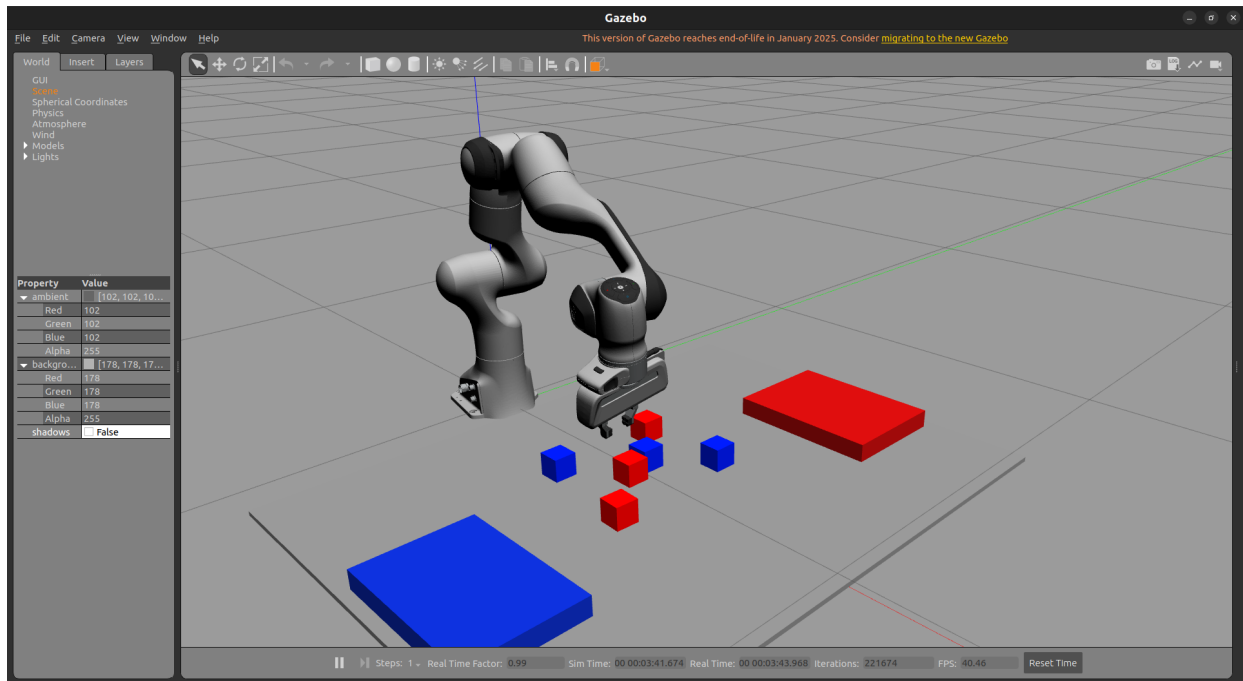
Platform: ROS 2 Humble, Gazebo Classic, MoveIt 2, Panda arm

Submission: complete ROS 2 package + README + short demonstration video

Goal

Create a ROS 2 Python package that commands the Panda robot to sort six objects in Gazebo:

- Move the three red boxes to the red container.
- Move the three blue boxes to the blue container.
- Use meaningful arm and gripper commands so the robot visibly performs the task.
- Leave the provided base package unchanged. Build your project package around it.



Expected scene: mixed red and blue objects with matching destination containers.

Get The Base Package

```
cd ~/ros2_ws/src
git clone https://github.com/ake1999/
mcg5353_panda_gazebo_moveit_pick_place_projects.git
```

```
cd ~/ros2_ws
colcon build --symlink-install \
  --packages-select mcg5353_panda_gazebo_moveit
source install/setup.bash
```

Run the prepared Panda Gazebo + MoveIt stack:

```
ros2 launch mcg5353_panda_gazebo_moveit \
  panda_moveit_gazebo.launch.py
```

Before writing your project logic, confirm that Panda remains upright, RViz opens, MoveIt is ready, and all three controllers are active.

Create Your Project Package

Create a separate package. You may choose a different valid package name.

```
cd ~/ros2_ws/src
ros2 pkg create --build-type ament_python \
  mcg5353_panda_sorting_project \
  --dependencies rclpy control_msgs trajectory_msgs \
  sensor_msgs geometry_msgs moveit_msgs
```

Your package should include:

- `package.xml`, `setup.py`, and `setup.cfg`;
- a Python node such as `sorting_node.py`;
- a console-script entry point so the node runs with `ros2 run`;
- a README containing exact build, launch, and run commands.

Required Task

The objects are named:

`red_box_1`, `red_box_2`, `red_box_3`, `blue_box_1`, `blue_box_2`, `blue_box_3`.

The destinations are `red_container` and `blue_container`.

For each box, your task sequence should clearly include:

1. move to a safe approach pose;
2. open the gripper;
3. approach the selected box;
4. close the gripper or clearly apply your grasp/attachment logic;
5. move toward the matching container;
6. release the box;
7. retreat to a safe pose.

Develop incrementally: make one object work, then generalize the sequence to all six objects. You may use MoveIt pose goals, controller joint goals, or a well-explained combination.

Useful Interfaces

Interface	Purpose
/move_action	Ask MoveIt to plan and move toward a new robot pose.
/panda_arm_controller/ follow_joint_trajectory	Execute known arm joint trajectories.
/panda_hand_controller/ gripper_cmd	Open or close the Panda gripper.
/joint_states	Read current joint positions and velocities.
gz model -m <name> -p	Inspect a Gazebo model pose while debugging.

Required Demonstration

Your video must show:

1. the command used to launch the base package;
2. Panda, both containers, and all six mixed-color boxes in Gazebo;
3. your node started with `ros2 run`;
4. visible arm and gripper motion;
5. all red boxes in the red container;
6. all blue boxes in the blue container;
7. one debug check, such as `ros2 control list_controllers`.

Grading: 100 Project Points

Your score out of 100 is converted to the 15% course weight.

Criterion	Points
Package builds, installs, and runs using documented commands	10
Correct integration with the provided Gazebo + MoveIt base package	15
Arm and gripper commands are used meaningfully and safely	20
All three red boxes reach the red container	15
All three blue boxes reach the blue container	15
Sequence is repeatable and handles all six objects clearly	10
Code organization, names, comments, and readable task logic	5
README and complete video evidence	10
Total	100

Optional Computer-Vision Bonus: Up To +5 Project Points

This is an advanced, optional extension. The bonus applies only to the combined project component of the course (Project 1 + Project 2, worth 25%). It does not add marks to theory, exams, or other course components.

For the bonus, add a simulated camera and use image data to decide whether an object is red or blue before choosing its destination. A reasonable approach is:

- add an RGB camera sensor and publish an image topic;
- subscribe to the image in your Python node;
- use a simple color-space threshold or segmentation method;
- connect the detected color to your sorting decision;
- show the camera image, detection result, and correct placement in the video.

The bonus requires working computer-vision evidence and an explanation in the README. Merely adding a camera model is not enough. The maximum bonus is five percentage points on the Project 2 score, capped according to course policy.

Submission Checklist

- Complete Project 2 ROS 2 package;
- no unfinished `NotImplementedError`;
- README with build, launch, run, and dependency instructions;
- short video proving the required result;
- optional computer-vision files and explanation, if attempted.